

EXPERIMENT 15:Decision Tree Algorithm for Gaming

Aim

To implement a Decision Tree algorithm for game playing and use it to make the best decision based on the given game conditions.

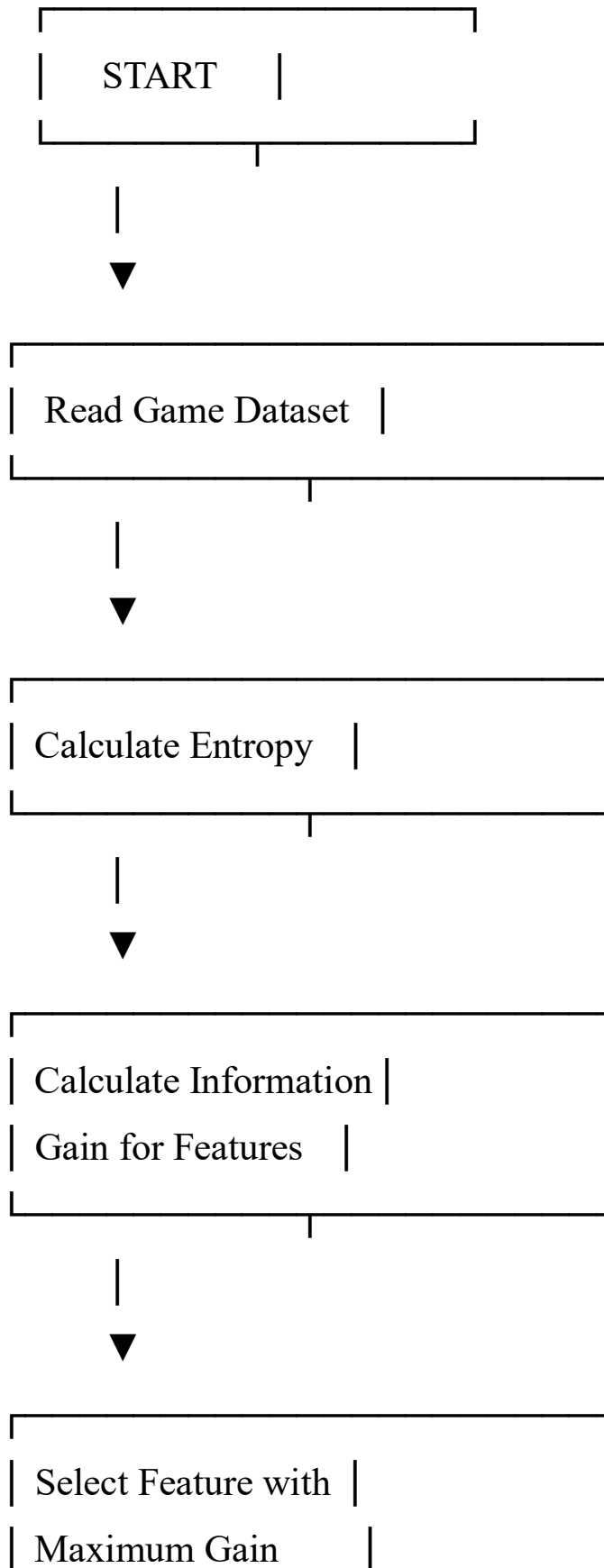
Objective

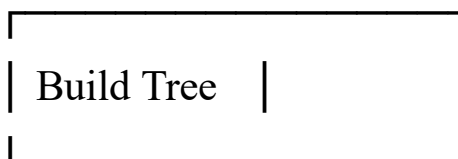
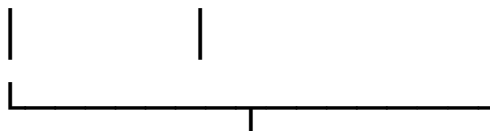
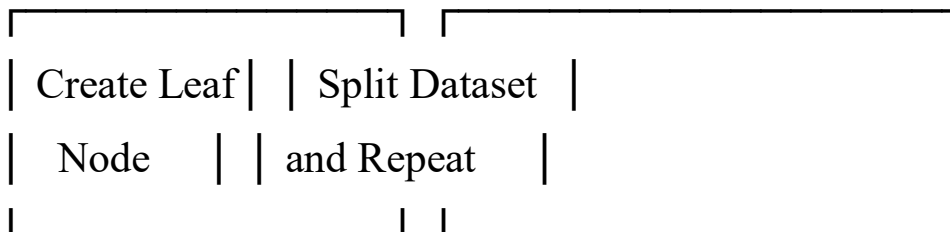
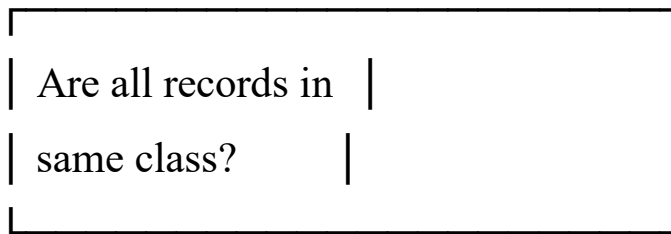
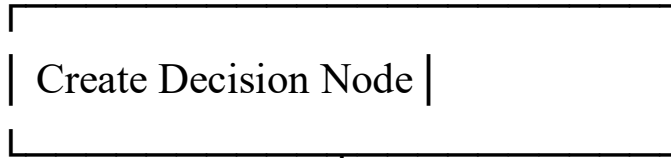
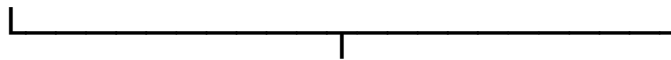
- To understand the concept of Decision Trees.
- To represent game decisions in a tree structure.
- To evaluate different possible choices.
- To select the best decision based on the given conditions.

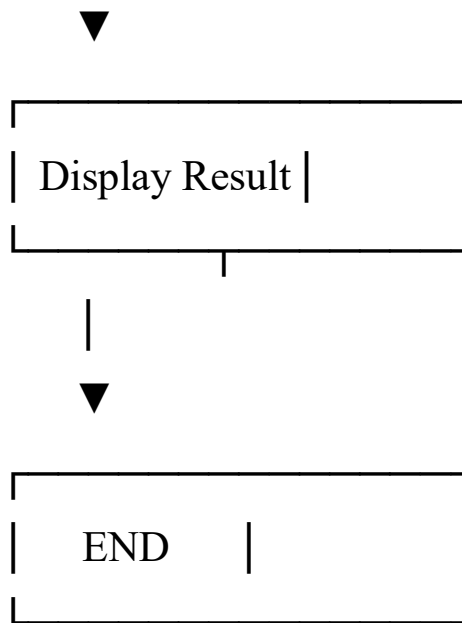
Algorithm

5. Start the program.
6. Create the game data containing different conditions and decisions.
7. Calculate the entropy of the dataset.
8. Calculate the information gain for each available feature.
9. Select the feature having the highest information gain as the decision node.
10. Divide the dataset according to the selected feature.
11. Repeat the process for each subset.
12. Stop when all records in a subset belong to the same class or no features remain.
13. Construct the complete Decision Tree.
14. Use the tree to make a decision for a new input.
15. Display the result.
16. Stop.

Flowchart







Code

```
# Decision Tree using ID3 Algorithm
```

```
import math
```

```
from collections import Counter
```

```
# Calculate Entropy
```

```
def entropy(data):
```

```
    labels = [row[-1] for row in data]
```

```
    count = Counter(labels)
```

```
    total = len(labels)
```

```
    ent = 0
```

```
    for value in count.values():
```

```
probability = value / total  
ent -= probability * math.log2(probability)
```

```
return ent
```

```
# Calculate Information Gain
```

```
def information_gain(data, index):
```

```
    total_entropy = entropy(data)
```

```
    values = set(row[index] for row in data)
```

```
    weighted_entropy = 0
```

```
    for value in values:
```

```
        subset = [row for row in data if row[index] == value]
```

```
        weighted_entropy += (
```

```
            len(subset) / len(data)
```

```
        ) * entropy(subset)
```

```
    return total_entropy - weighted_entropy
```

```
# Build Decision Tree
```

```
def build_tree(data, features):
```

```
labels = [row[-1] for row in data]
```

```
# If all labels are the same
```

```
if len(set(labels)) == 1:
```

```
    return labels[0]
```

```
# If no features remain
```

```
if not features:
```

```
    return Counter(labels).most_common(1)[0][0]
```

```
# Find feature with maximum information gain
```

```
gains = [information_gain(data, i) for i in features]
```

```
best_index = features[gains.index(max(gains))]
```

```
tree = {best_index: {}}
```

```
# Get possible values
```

```
values = set(row[best_index] for row in data)
```

```
for value in values:
```

```
    subset = [
```

```
        row for row in data
```

```
        if row[best_index] == value
    ]
```

```
    remaining_features = [
        i for i in features
        if i != best_index
    ]
```

```
    tree[best_index][value] = build_tree(
        subset,
        remaining_features
    )
```

```
    return tree
```

```
# Game Dataset
```

```
data = [
    ['Sunny', 'Hot', 'No'],
    ['Sunny', 'Cool', 'Yes'],
    ['Rainy', 'Cool', 'Yes'],
    ['Rainy', 'Hot', 'No'],
    ['Overcast', 'Cool', 'Yes'],
    ['Overcast', 'Hot', 'Yes']
]
```

```
# Feature names
features = ['Weather', 'Temperature']

# Feature indexes
feature_indexes = [0, 1]

# Build Decision Tree
tree = build_tree(data, feature_indexes)

print("Decision Tree:")
print(tree)
```

Output

Decision Tree:

```
{0: {'Sunny': {1: {'Hot': 'No', 'Cool': 'Yes'}},
      'Rainy': {1: {'Hot': 'No', 'Cool': 'Yes'}},
      'Overcast': 'Yes'}}
```

Output:


```
main.py
70 data = [
71     ['Sunny', 'Hot', 'No'],
72     ['Sunny', 'Cool', 'Yes'],
73     ['Rainy', 'Cool', 'Yes'],
74     ['Rainy', 'Hot', 'No'],
75     ['Overcast', 'Cool', 'Yes'],
76     ['Overcast', 'Hot', 'Yes']
77 ]
78
79 # Feature names
80 features = ['Weather', 'Temperature']
81
82 # Feature indexes
83 feature_indexes = [0, 1]
84
85 # Build the decision tree
86 tree = build_tree(data, feature_indexes)
87
88 print("Decision Tree:")
89 print(tree)
90
```

input

```
Decision Tree:
1: {'Hot': {0: {'Overcast': 'Yes', 'Rainy': 'No', 'Sunny': 'No'}}, 'Cool': 'Yes'}}
```

Result

The Decision Tree algorithm using ID3 was successfully implemented. The algorithm calculated Entropy and Information Gain and constructed a decision tree from the given game-related dataset.

Conclusion

The Decision Tree algorithm provides an effective way to represent decisions in a game. By selecting the feature with the highest information gain at each step, the algorithm constructs a tree that can be used to make decisions or classify game situations.